

Ansible Complete Reference Guide for Linux

Table of Contents

1. [Playbook Structure & Execution Flow](#)
 2. [Playbook Sections & Hooks \(Hierarchy\)](#)
 3. [Magic Variables](#)
 4. [Ansible Commands](#)
 5. [Modules by Purpose](#)
-

Playbook Structure & Execution Flow

```
---
# Playbook Root
- name: Play Name
  hosts: target_group

  # Pre-execution sections (in order)
  gather_facts: yes/no
  vars:
  vars_files:
  vars_prompt:

  # Role/Task inclusion
  roles:
  pre_tasks:
  tasks:
  post_tasks:
  handlers:

  # Execution control
  serial:
  max_fail_percentage:
  any_errors_fatal:
```

Playbook Sections & Hooks (Hierarchy)

Execution Order (Top to Bottom)

1. Connection & Setup

- `connection:` - Transport method (ssh, local, docker, etc.)
- `remote_user:` - User to connect as
- `become:` - Enable privilege escalation
- `become_user:` - User to become (default: root)
- `become_method:` - Method for privilege escalation (sudo, su, pbrun, etc.)

2. Variable Loading (precedence low to high)

- `role defaults` - Lowest priority variables
- `inventory vars` - Variables from inventory files
- `inventory group_vars` - Group-specific variables
- `inventory host_vars` - Host-specific variables
- `playbook vars_files:` - External variable files
- `playbook vars:` - Inline playbook variables
- `vars_prompt:` - Interactive user input
- `registered vars` - Variables from task output
- `set_facts` - Facts set during execution
- `extra_vars` - Command line vars (-e) - HIGHEST priority

3. Fact Gathering

- `gather_facts:` - Collect system information (yes/no)
- `fact_caching:` - Cache gathered facts

4. Pre-Task Phase

- `pre_tasks:` - Tasks run before roles

5. Role Phase (per role)

- `roles:` - Role inclusion
 - Role `tasks/main.yml` - Main role tasks
 - Role `handlers/main.yml` - Role-specific handlers

6. Main Task Phase

- `tasks:` - Primary playbook tasks

7. Post-Task Phase

- `post_tasks:` - Tasks run after main tasks

8. Handler Phase (triggered)

- `handlers:` - Event-driven tasks (notified during execution)

Task-Level Directives

- `name:` - Human-readable task description
- `register:` - Store task output to variable
- `when:` - Conditional task execution
- `loop:` / `with_*:` - Iteration constructs
- `notify:` - Trigger handler execution
- `changed_when:` - Override changed status
- `failed_when:` - Define failure conditions
- `ignore_errors:` - Continue on task failure
- `delegate_to:` - Run task on different host
- `run_once:` - Execute task only once across all hosts
- `tags:` - Task categorization for selective execution
- `block:` - Group tasks for error handling
- `rescue:` - Tasks on block failure
- `always:` - Tasks that always run

Play-Level Directives

- `hosts:` - Target host pattern
 - `serial:` - Rolling execution batch size
 - `max_fail_percentage:` - Failure tolerance threshold
 - `any_errors_fatal:` - Abort on any failure
 - `strategy:` - Execution strategy (linear, free, debug)
 - `order:` - Host execution order (inventory, reverse_inventory, sorted, shuffle)
 - `environment:` - Environment variables for all tasks
 - `no_log:` - Suppress task output logging
 - `throttle:` - Limit concurrent task execution
-

Magic Variables

Host/Inventory Variables

- `inventory_hostname` - Name of current host in inventory
- `inventory_hostname_short` - Short hostname (before first dot)
- `inventory_dir` - Directory containing inventory file
- `inventory_file` - Inventory file path
- `groups` - Dictionary of all groups and their hosts
- `group_names` - List of groups current host belongs to
- `hostvars` - Dictionary of variables for all hosts
- `play_hosts` - List of hosts in current play
- `ansible_play_batch` - List of hosts in current batch

Connection Variables

- `ansible_host` - Actual hostname/IP to connect to
- `ansible_port` - SSH port to use
- `ansible_user` - Remote user for connection
- `ansible_connection` - Connection type (ssh, local, docker)
- `ansible_ssh_private_key_file` - Private key path
- `ansible_become` - Enable privilege escalation (boolean)
- `ansible_become_user` - User to become
- `ansible_become_method` - Escalation method

Ansible Environment

- `ansible_version` - Ansible version information
- `ansible_playbook_python` - Python interpreter on control node
- `playbook_dir` - Directory containing current playbook
- `role_path` - Path to current role
- `role_name` - Name of current role

Gathered Facts (with `gather_facts: yes`)

- `ansible_distribution` - OS distribution (Ubuntu, CentOS, etc.)

- `ansible_distribution_version` - OS version number
- `ansible_os_family` - OS family (Debian, RedHat, etc.)
- `ansible_kernel` - Kernel version
- `ansible_architecture` - System architecture (x86_64, etc.)
- `ansible_processor_cores` - CPU core count
- `ansible_memtotal_mb` - Total RAM in MB
- `ansible_hostname` - System hostname
- `ansible_fqdn` - Fully qualified domain name
- `ansible_default_ipv4.address` - Primary IPv4 address
- `ansible_default_ipv6.address` - Primary IPv6 address
- `ansible_all_ipv4_addresses` - List of all IPv4 addresses
- `ansible_interfaces` - List of network interfaces
- `ansible_mounts` - List of mounted filesystems
- `ansible_devices` - Block device information
- `ansible_env` - Environment variables on remote host
- `ansible_date_time` - Current date/time information
- `ansible_python_version` - Python version on target
- `ansible_selinux` - SELinux status and mode

Task Execution

- `item` - Current loop iteration value
 - `ansible_loop` - Loop metadata (index, first, last, etc.)
 - `omit` - Placeholder to omit module parameter
 - `ansible_check_mode` - Boolean for check mode (--check)
 - `ansible_diff_mode` - Boolean for diff mode (--diff)
 - `ansible_verbosity` - Current verbosity level
-

Ansible Commands

Core Commands

- `ansible` - Run ad-hoc commands on remote hosts

- `ansible-playbook` - Execute Ansible playbooks
- `ansible-vault` - Encrypt/decrypt sensitive files
- `ansible-galaxy` - Manage roles and collections
- `ansible-inventory` - Display inventory information
- `ansible-config` - View/manage Ansible configuration
- `ansible-doc` - Display module documentation
- `ansible-pull` - Pull playbooks from VCS and execute
- `ansible-console` - Interactive REPL for ad-hoc tasks

ansible (Ad-Hoc)

```
ansible <pattern> -m <module> -a "<args>" [options]

# Key Options
-i INVENTORY      Specify inventory file/directory
-m MODULE         Module to execute (default: command)
-a ARGS           Module arguments
-b, --become      Run with privilege escalation
-u USER          Remote user
-k               Ask for SSH password
-K               Ask for privilege escalation password
-e VARS           Extra variables (key=value)
-f FORKS          Parallel process count (default: 5)
-t TREE           Save output to directory
--check           Dry-run mode
--diff            Show file differences
-l SUBSET         Limit to subset of hosts
-v, -vv, -vvv    Increase verbosity
```

ansible-playbook

```
ansible-playbook playbook.yml [options]

# Key Options
-i INVENTORY      Specify inventory
-e VARS           Extra variables
-t TAGS           Run only tagged tasks
--skip-tags TAGS  Skip tagged tasks
--start-at-task   Start at specific task
--step            Confirm each task before running
```

```
--check          Dry-run mode
--diff           Show file differences
-l SUBSET        Limit to subset of hosts
-f FORKS         Parallel process count
--syntax-check   Validate playbook syntax
--list-tasks     List all tasks
--list-hosts     List matching hosts
--list-tags      List available tags
-v, -vv, -vvv   Increase verbosity
```

ansible-vault

```
ansible-vault <subcommand> file.yml [options]
```

Subcommands

```
create          Create new encrypted file
edit            Edit encrypted file
encrypt         Encrypt existing file
decrypt         Decrypt file
view            View encrypted file contents
rekey           Change encryption password
encrypt_string  Encrypt single value
```

Options

```
--vault-id LABEL  Vault identity label
--ask-vault-pass  Prompt for password
--vault-password-file FILE Password file path
```

ansible-galaxy

```
ansible-galaxy <subcommand> [options]
```

Role Subcommands

```
init NAME        Create new role structure
install NAME      Install role from Galaxy
remove NAME       Remove installed role
list             List installed roles
search TERM       Search Galaxy for roles
info NAME         Display role information
```

Collection Subcommands

```
collection install  Install collection
```

```
collection list      List installed collections
collection init      Create collection skeleton
collection build     Build collection tarball
collection publish   Publish to Galaxy

# Options
-p PATH              Role/collection installation path
-r FILE              Requirements file
--force              Force overwrite
```

ansible-inventory

```
ansible-inventory [options]

# Key Options
-i INVENTORY         Specify inventory
--list               List all hosts (JSON)
--graph              Display inventory tree
--host HOST          Show host variables
-y, --yaml           Output in YAML format
--toml               Output in TOML format
```

ansible-config

```
ansible-config <subcommand> [options]

# Subcommands
list                 List all configuration options
dump                 Show current configuration
view                 View configuration file
init                 Create initial config file

# Options
-c CONFIG            Config file path
--type TYPE          Config type (all, base, etc.)
```

ansible-doc

```
ansible-doc <module_name> [options]
```



```
# Options
-l, --list      List available modules
-s, --snippet  Show module snippets
-t TYPE        Documentation type (module, plugin, etc.)
--metadata     Show module metadata
-j            JSON output
```

Modules by Purpose

PACKAGE MANAGEMENT

apt (Debian/Ubuntu)

- `apt` - Manage apt packages and repositories

yum (RHEL/CentOS 7)

- `yum` - Manage packages with yum

dnf (RHEL/CentOS 8+)

- `dnf` - Manage packages with dnf

package (Generic)

- `package` - Generic OS package manager (auto-detects)

pip (Python)

- `pip` - Manage Python packages

gem (Ruby)

- `gem` - Manage Ruby gems

npm (Node.js)

- `npm` - Manage Node.js packages

snap

- `snap` - Manage snap packages

flatpak

- `flatpak` - Manage Flatpak packages

apt_repository

- `apt_repository` - Add/remove APT repositories

yum_repository

- `yum_repository` - Add/remove YUM repositories

apt_key

- `apt_key` - Manage APT repository keys

rpm_key

- `rpm_key` - Manage RPM package signing keys

FILE OPERATIONS

file

- `file` - Manage file attributes (permissions, ownership, state)

copy

- `copy` - Copy files from local to remote

template

- `template` - Process Jinja2 templates to remote files

fetch

- `fetch` - Retrieve files from remote to local

lineinfile

- `lineinfile` - Ensure specific line exists/absent in file

blockinfile

- `blockinfile` - Insert/update/remove text blocks in files

replace

- `replace` - Replace text in files using regex

find

- `find` - Search for files matching criteria

stat

- `stat` - Retrieve file/filesystem status

synchronize

- `synchronize` - Sync files using rsync

assemble

- `assemble` - Assemble files from fragments

patch

- `patch` - Apply diff files to files

unarchive

- `unarchive` - Extract archive files

archive

- `archive` - Create compressed archives

iso_extract

- `iso_extract` - Extract files from ISO images

acl

- `acl` - Manage POSIX ACLs

read_csv

- `read_csv` - Read CSV files into variables
-

SYSTEM OPERATIONS

service

- `service` - Manage system services (generic)

systemd

- `systemd` - Manage systemd services and units

sysvinit

- `sysvinit` - Manage SysV init services

command

- `command` - Execute commands (no shell processing)

shell

- `shell` - Execute commands through shell

script

- `script` - Transfer and execute local script

raw

- `raw` - Execute raw SSH commands (no Python needed)

reboot

- `reboot` - Reboot system with wait handling

hostname

- `hostname` - Manage system hostname

timezone

- `timezone` - Configure system timezone

locale_gen

- `locale_gen` - Generate system locales

sysctl

- `sysctl` - Manage kernel parameters

kernel_blacklist

- `kernel_blacklist` - Blacklist kernel modules

modprobe

- `modprobe` - Load/unload kernel modules

capabilities

- `capabilities` - Manage Linux capabilities

pam_limits

- `pam_limits` - Manage PAM limits configuration

selinux

- `selinux` - Manage SELinux mode and policy

selinux_permissive

- `selinux_permissive` - Set SELinux domain to permissive

seboolean

- `seboolean` - Manage SELinux booleans

seport

- `seport` - Manage SELinux port type definitions

sefcontext

- `sefcontext` - Manage SELinux file context mappings

selogin

- `selogin` - Manage Linux user to SELinux user mapping
-

USER & GROUP MANAGEMENT

user

- `user` - Manage user accounts

group

- `group` - Manage groups

authorized_key

- `authorized_key` - Manage SSH authorized_keys

known_hosts

- `known_hosts` - Manage SSH known_hosts entries

pamd

- `pamd` - Manage PAM service configurations
-

STORAGE & FILESYSTEM

mount

- `mount` - Manage mount points

filesystem

- `filesystem` - Create filesystems

lv

- `lv` - Manage LVM volume groups

lvol

- `lvol` - Manage LVM logical volumes

parted

- `parted` - Manage disk partitions

crypttab

- `crypttab` - Manage encrypted block devices

gluster_volume

- `gluster_volume` - Manage GlusterFS volumes

zfs

- `zfs` - Manage ZFS filesystems

zfs_facts

- `zfs_facts` - Gather ZFS dataset information

zpool_facts

- `zpool_facts` - Gather ZFS pool information

NETWORK CONFIGURATION

hostname

- `hostname` - Set system hostname

nmcli

- `nmcli` - Manage NetworkManager connections

interfaces_file

- `interfaces_file` - Manage /etc/network/interfaces

debconf

- `debconf` - Configure Debian packages

firewalld

- `firewalld` - Manage firewalld rules

ufw

- `ufw` - Manage UFW firewall

iptables

- `iptables` - Manage iptables rules

seport

- `seport` - Manage SELinux network port type definitions
-

WEB & APPLICATION SERVERS

apache2_module

- `apache2_module` - Enable/disable Apache modules

apache2_mod_proxy

- `apache2_mod_proxy` - Manage Apache proxy configuration

htpasswd

- `htpasswd` - Manage Apache htpasswd files

supervisorctl

- `supervisorctl` - Manage supervisor programs

django_manage

- `django_manage` - Run Django management commands
-

DATABASE

mysql_db

- `mysql_db` - Manage MySQL databases

mysql_user

- `mysql_user` - Manage MySQL users and privileges

mysql_replication

- `mysql_replication` - Manage MySQL replication

postgresql_db

- `postgresql_db` - Manage PostgreSQL databases

postgresql_user

- `postgresql_user` - Manage PostgreSQL users

postgresql_privs

- `postgresql_privs` - Grant/revoke PostgreSQL privileges

mongodb_user

- `mongodb_user` - Manage MongoDB users

redis

- `redis` - Manage Redis instances

CONTAINERS

docker_container

- `docker_container` - Manage Docker containers

docker_image

- `docker_image` - Manage Docker images

docker_network

- `docker_network` - Manage Docker networks

docker_volume

- `docker_volume` - Manage Docker volumes

docker_compose

- `docker_compose` - Manage multi-container Docker applications

docker_swarm

- `docker_swarm` - Manage Docker Swarm cluster

docker_swarm_service

- `docker_swarm_service` - Manage Docker Swarm services

podman_container

- `podman_container` - Manage Podman containers

podman_image

- `podman_image` - Manage Podman images

buildah_image

- `buildah_image` - Build OCI/Docker images with Buildah

VIRTUALIZATION

virt (libvirt/KVM)

- `virt` - Manage libvirt virtual machines

virt_net

- `virt_net` - Manage libvirt networks

virt_pool

- `virt_pool` - Manage libvirt storage pools

proxmox

- `proxmox` - Manage Proxmox VE virtual machines

proxmox_kvm

- `proxmox_kvm` - Manage Proxmox KVM virtual machines

CLOUD - AWS

ec2_instance

- `ec2_instance` - Manage EC2 instances

ec2_vpc_net

- `ec2_vpc_net` - Manage AWS VPCs

ec2_vpc_subnet

- `ec2_vpc_subnet` - Manage VPC subnets

ec2_security_group

- `ec2_security_group` - Manage security groups

ec2_key

- `ec2_key` - Manage EC2 key pairs

s3_bucket

- `s3_bucket` - Manage S3 buckets

s3_object

- `s3_object` - Manage S3 objects

rds_instance

- `rds_instance` - Manage RDS database instances

route53

- `route53` - Manage Route53 DNS records

cloudformation

- `cloudformation` - Manage CloudFormation stacks

lambda

- `lambda` - Manage AWS Lambda functions

iam_user

- `iam_user` - Manage IAM users

iam_role

- `iam_role` - Manage IAM roles

iam_policy

- `iam_policy` - Manage IAM policies
-

CLOUD - AZURE

azure_rm_virtualmachine

- `azure_rm_virtualmachine` - Manage Azure virtual machines

azure_rm_resourcegroup

- `azure_rm_resourcegroup` - Manage resource groups

azure_rm_storageaccount

- `azure_rm_storageaccount` - Manage storage accounts

azure_rm_virtualmachinescaleset

- `azure_rm_virtualmachinescaleset` - Manage VM scale sets

azure_rm_networkinterface

- `azure_rm_networkinterface` - Manage network interfaces

azure_rm_securitygroup

- `azure_rm_securitygroup` - Manage network security groups
-

CLOUD - GCP

gcp_compute_instance

- `gcp_compute_instance` - Manage GCE instances

gcp_compute_disk

- `gcp_compute_disk` - Manage GCE persistent disks

gcp_compute_network

- `gcp_compute_network` - Manage GCE networks

gcp_compute_firewall

- `gcp_compute_firewall` - Manage GCE firewall rules

gcp_storage_bucket

- `gcp_storage_bucket` - Manage GCS buckets

gcp_sql_instance

- `gcp_sql_instance` - Manage Cloud SQL instances
-

CLOUD - OPENSTACK

os_server

- `os_server` - Manage OpenStack compute instances

os_network

- `os_network` - Manage OpenStack networks

os_subnet

- `os_subnet` - Manage OpenStack subnets

os_security_group

- `os_security_group` - Manage security groups

os_volume

- `os_volume` - Manage Cinder volumes

os_image

- `os_image` - Manage Glance images
-

MONITORING & LOGGING

nagios

- `nagios` - Perform maintenance actions on Nagios

datadog_monitor

- `datadog_monitor` - Manage Datadog monitors

zabbix_host

- `zabbix_host` - Manage Zabbix hosts

logstash_plugin

- `logstash_plugin` - Manage Logstash plugins

elasticsearch_plugin

- `elasticsearch_plugin` - Manage Elasticsearch plugins
-

VERSION CONTROL

git

- `git` - Deploy software from git repositories

git_config

- `git_config` - Manage git configuration

github_key

- `github_key` - Manage GitHub access keys

github_release

- `github_release` - Manage GitHub releases

gitlab_project

- `gitlab_project` - Manage GitLab projects

gitlab_user

- `gitlab_user` - Manage GitLab users
-

MESSAGING

rabbitmq_user

- `rabbitmq_user` - Manage RabbitMQ users

rabbitmq_vhost

- `rabbitmq_vhost` - Manage RabbitMQ virtual hosts

rabbitmq_queue

- `rabbitmq_queue` - Manage RabbitMQ queues

rabbitmq_plugin

- `rabbitmq_plugin` - Manage RabbitMQ plugins
-

NOTIFICATION

mail

- `mail` - Send email notifications

slack

- `slack` - Send Slack messages

telegram

- `telegram` - Send Telegram messages

irc

- `irc` - Send IRC messages

jabber

- `jabber` - Send Jabber/XMPP messages
-

SOURCE CONTROL

subversion

- `subversion` - Deploy from Subversion repositories

hg

- `hg` - Deploy from Mercurial repositories

bzr

- `bzr` - Deploy from Bazaar repositories
-

UTILITIES

debug

- `debug` - Print debugging messages

assert

- `assert` - Assert conditions are met

fail

- `fail` - Fail with custom message

pause

- `pause` - Pause playbook execution

wait_for

- `wait_for` - Wait for condition before continuing

wait_for_connection

- `wait_for_connection` - Wait for system to be reachable

async_status

- `async_status` - Check async task status

set_fact

- `set_fact` - Set host variable during execution

set_stats

- `set_stats` - Set statistics for playbook run

include_vars

- `include_vars` - Load variables from files

include_tasks

- `include_tasks` - Include task file dynamically

include_role

- `include_role` - Include role dynamically

import_tasks

- `import_tasks` - Import task file statically

import_role

- `import_role` - Import role statically

import_playbook

- `import_playbook` - Import another playbook

add_host

- `add_host` - Add host to inventory during execution

group_by

- `group_by` - Create dynamic groups based on facts

meta

- `meta` - Execute Ansible actions (flush_handlers, refresh_inventory, etc.)
-

ENCRYPTION

openssl_certificate

- `openssl_certificate` - Generate/renew OpenSSL certificates

openssl_privatekey

- `openssl_privatekey` - Generate OpenSSL private keys

openssl_publickey

- `openssl_publickey` - Generate OpenSSL public keys

openssl_csr

- `openssl_csr` - Generate OpenSSL certificate signing requests

acme_certificate

- `acme_certificate` - Create SSL certificates with ACME (Let's Encrypt)
-

TESTING

uri

- `uri` - Interact with HTTP/HTTPS services

get_url

- `get_url` - Download files from HTTP/HTTPS/FTP

slurp

- `slurp` - Read file from remote and base64 encode

expect

- `expect` - Execute commands with expect-like prompts

setup

- `setup` - Gather facts explicitly

gather_facts

- `gather_facts` - Gather facts module (explicit call)
-

INVENTORY

add_host

- `add_host` - Add host to in-memory inventory

group_by

- `group_by` - Create groups based on variables
-

WINDOWS (Informational - Linux focus)

- `win_*` modules - Windows-specific modules (200+ available)
-

Quick Reference Commands

```
# List all modules
ansible-doc -l

# View specific module documentation
ansible-doc <module_name>

# Test playbook syntax
ansible-playbook playbook.yml --syntax-check

# Dry run (check mode)
ansible-playbook playbook.yml --check

# Run specific tags
ansible-playbook playbook.yml -t "setup,deploy"

# Skip tags
ansible-playbook playbook.yml --skip-tags "cleanup"

# Limit to specific hosts
ansible-playbook playbook.yml -l "webservers"

# Step through tasks
ansible-playbook playbook.yml --step

# Verbose output
ansible-playbook playbook.yml -vvv
```

Common Patterns

Loop Variables

```
# Standard loop
- name: Install packages
  apt:
    name: "{{ item }}"
  loop:
    - nginx
```

```
- postgresql

# Loop with dict
- name: Create users
  user:
    name: "{{ item.name }}"
    groups: "{{ item.groups }}"
  loop:
    - { name: 'alice', groups: 'wheel' }
    - { name: 'bob', groups: 'users' }
```

Conditionals

```
# Basic when
- name: Install on Ubuntu
  apt:
    name: nginx
  when: ansible_distribution == "Ubuntu"

# Multiple conditions
- name: Complex condition
  service:
    name: httpd
  when:
    - ansible_os_family == "RedHat"
    - ansible_distribution_major_version == "8"
```

Error Handling

```
# Block with rescue
- block:
    - name: Risky operation
      command: /bin/risky
  rescue:
    - name: Handle failure
      debug:
        msg: "Operation failed"
  always:
    - name: Cleanup
      file:
```

```
path: /tmp/lock  
state: absent
```

Last Updated: February 2026

Ansible Version Reference: 2.15+